Program:

```
def encode(d):
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h.
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*1 + c4*4.
    if error_pos != 0:
        Print(f"Error detected at bit
position: {error_pos} -> (orrecting ...")
        h[error_pos - 1] ^= 1
    else:
        Print("NO error detected")
    return error_pos.
def to_str(bits)
    return "".join(str(b) for b in bits)
def Main():
    print("== HAMMING CODE - ERROR
            detection and correction ==")
    n = int(input("Enter number of 4 bit
            data blocks in the stream: "))
```

Ex no: 6

Date: 8/8/26 Error Detection and Correction using Hamming Code.

Aim:

To write a program to implement error detection and correction using the Hamming Code.

Algorithm:

1. Start the Program.
2. Read the number of 4 bit data blocks and the input data bits
3. Arrange the data and parity bits in a 7-bit hamming code.
4. calculate the Parity bits using even parity and generate the hamming code.
5. Induce a single bit error at the specific Position
6. At the reciever calculate the Syndrome to identify the error Position.
7. Correct the error by inverting the faulty bit.
8. Extract the and display the recovered original data bits

```
for b in range (1, n+1):
    Print(f"\n--- block {b} ---")
    d = list(map(int, input("Enter 4 data bits
    (Space seperated, eg 1 0 1 1): ").split())
    h = encode(d)
    Print(f"original Data Bits : {to_str(d)}")
    Print(f" Encoded Hamming code (7 bits):
                {to_str(h)}")

Pos = int(input("Enter bit position to
    Introduce error (1-7, 0, for no error): "))
If 1 <= Pos <= 7:
    h[Pos-1] = 1
    Print(f"transmitted code
    {with error induced at pos {Pos}):
                {to_str(h)}")
else:
    print(f"transmitted code (no error
                induced): {to_str(h)}")
    Print(f"Recieved code: {to_str(h)}")
    Print(f"Corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    Print(f"Recovered data bits: {to_str(rd)}")
if name == "__Main__":
    Main()
```

OUTPUT

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1011): 1011
Original Data Bits     : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code          : 0100011
Checking for errors...
  Error detected at bit position: 3 -> Correcting...
Corrected Code         : 0110011
Recovered Data Bits    : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1011): 0110
Original Data Bits     : 0110
```



```
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code          : 1100100
Checking for errors...
  Error detected at bit position: 6 -> Correcting...
Corrected Code         : 1100110
Recovered Data Bits    : 0110
```

Result:

The Programming to Implement error detection and correction using Hamming code Concept was Successfully written in Python and executed. verified using a sample input data stream.